KLUDGE WAGON HAYWIRE · SR-404 SERVICE CARD

C Preprocessor & Everyday Slips

Macros, headers, switches and one-character typos: language constraints, diagnostic scope and explicit control-flow policies.

Debug build:
`-std=c17 -Wall -Wextra -g -fsanitize=address,undefined`
Also try: `-O2 · -fanalyzer · -fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▸ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 13 entries

C-PP-01 Macro without parentheses

SR-404 · Turbo boost calc

BUGGY
```c
#define SQUARE(x) x * x
int boost(int rpm) {
    return SQUARE(rpm + 1);   // boost(3): expansion yields 7
}
```

▸ prints: boost=7

What goes wrong: Macro replacement operates on preprocessing tokens. The unparenthesized expansion is rpm + 1 * rpm + 1; boost(3) yields 7. Parentheses fix grouping but not repeated evaluation or signed overflow.

FIX
```c
#define SQUARE(x) ((x) * (x))   // if it must be a macro
static inline int square(int x) { return x * x; }  // typed; still requires representable product
int boost(int rpm) {
    return square(rpm + 1);   // 16
}
```

Avoid it: Author guidance: prefer a typed function when suitable. The shown square requires x*x to fit int, and boost also requires rpm+1 to fit. Parenthesize expression-macro operands and result; GCC14 documents -E for inspecting preprocessing. C const objects are not interchangeable with enum integer constant expressions.

Sources: C11 draft 6.10.3; 6.5p5; 6.6; GCC14 preprocessor options -E

C-PP-02 Macro evaluates an argument twice

SR-404 · Sensor FIFO

BUGGY
```c
#define MAX(a, b) ((a) > (b) ? (a) : (b))
int next_sample(void);            // pops the sensor FIFO
int peak(int limit) {
    return MAX(next_sample(), limit);  // may pop the FIFO twice
}
```

▸ prints: peak=102 calls=2

What goes wrong: The conditional operator evaluates its condition and only the selected operand. Here a sample above limit causes a second call; a sample not above limit is consumed only once. Repeated tokens do not imply every occurrence executes. The supplied driver shows the two-call branch.

FIX
```c
static inline int max_int(int a, int b) { return a > b ? a : b; }
int peak(int limit) {
    return max_int(next_sample(), limit);  // evaluated exactly once
}
```

Avoid it: Author guidance: inspect an expression macro before passing side effects. A function evaluates each argument once, but C does not specify the order among arguments. Keep interdependent side effects in separate statements. C11 _Generic selects by type; its controlling expression is not evaluated.

Sources: C11 draft 6.10.3.1; 6.5.15; 6.5.2.2; 6.5.1.1

C-PP-03 Multi-statement macro without do/while(0)

SR-404 · Gauge cluster reset

BUGGY
```c
#define RESET(g) (g).speed = 0; (g).rpm = 0
void on_fault(struct gauge *g, bool fault) {
    if (fault)
        RESET(*g);          // only .speed is conditional
}
```

▸ prints: speed=50 rpm=0

What goes wrong: In this expansion only the speed assignment is the if body; the rpm assignment runs even when fault is false. A compound-block macro followed by the caller's semicolon can separate an else from its if. do/while(0) packages a statement-like macro.

FIX
```c
#define RESET(g) do { (g).speed = 0; (g).rpm = 0; } while (0)
void on_fault(struct gauge *g, bool fault) {
    if (fault)
        RESET(*g);          // one statement; needs its ';'
}
```

Avoid it: Author guidance: prefer braces and typed helpers when practical. The do/while wrapper does not prevent repeated evaluation of g: this macro still uses it twice. Pass a stable valid gauge lvalue; adding break/continue inside such a macro needs separate control-flow review.

Sources: C11 draft 6.8.4.1; 6.8.5.2; 6.10.3

C-PP-04 Header without include guard

SR-404 · Engine computer modules (Ep 10)

BUGGY
```c
/* gauge.h */
struct gauge { int value; };

/* dash.c */
#include "gauge.h"
#include "gauge.h"   // repeated inclusion in this translation unit
```

▸ gcc error: redefinition of 'struct gauge'

What goes wrong: This header repeats a complete definition of the same struct tag in one scope when included twice, violating C's type-definition constraint. Repeating a compatible declaration is not always invalid; it is this repeated definition that requires protection.

FIX
```c
/* gauge.h */
#ifndef KWH_GAUGE_H
#define KWH_GAUGE_H
struct gauge { int value; };
#endif /* KWH_GAUGE_H */
```

Avoid it: Author guidance: use a unique project-prefixed guard for a header meant for repeated inclusion. All identifiers beginning with underscore plus uppercase, or double underscore, are reserved for any use. The #ifndef/#define mechanism is specified by ISO C; implementation-specific alternatives need their own documentation.

Sources: C11 draft 6.7.2.3p1; 6.10.1; 7.1.3

C-PP-05 = instead of ==

SR-404 · Park interlock

BUGGY
```c
void check_park(int gear) {
    if (gear = 0)          // assigns 0: never true
        engage_park();
}
```

▸ gcc -Wparentheses: suggest parentheses around assignment used as tru…

What goes wrong: Assignment yields the value stored after conversion. Here assigning int zero makes the condition false and changes the local parameter. A nonzero value is true after conversion; a nonzero source constant need not remain nonzero for every destination type.

FIX
```c
void check_park(const int gear) {  // assignment to const violates a constraint
    if (gear == 0)
        engage_park();
}
```

Avoid it: GCC14 documents -Wparentheses in -Wall for selected assignment-in-condition cases. A const-qualified local parameter makes this assignment violate a modifiable-lvalue constraint. Author guidance: express deliberate assignment and comparison clearly; warnings are not proof of the intended condition.

Sources: C11 draft 6.5.16; 6.8.4.1; GCC14 docs -Wparentheses; -Wall

C-PP-06 Switch fall-through

SR-404 · Brakes / ABS modes (Ep 4)

✕ BUGGY

```
enum brake_mode { ABS_ON, ABS_OFF };
int pressure(enum brake_mode m) {
    int p = 0;
    switch (m) {
    case ABS_ON:  p = 80;     // missing break: falls into
ABS_OFF
    case ABS_OFF: p = 100; break;
    }
    return p;
}
```

▸ `gcc -Wimplicit-fallthrough=: this statement may fall through`

What goes wrong: After the switch jumps to ABS_ON, execution continues to the next statement and then the ABS_OFF label because no break/return intervenes. This function returns the integer100; pressure units and real ABS behavior are a teaching model, not a vehicle specification.

Sources: C11 draft 6.8.4.2; 6.8.6.3; GCC14 docs -Wimplicit-fallthrough; -Wswitch; -Wswitch-enum

✓ FIX

```
enum brake_mode { ABS_ON, ABS_OFF };
int pressure(enum brake_mode m) {
    switch (m) {
    case ABS_ON:  return 80;
    case ABS_OFF: return 100;
    }
    return 0;   // GCC -Wswitch can warn about omitted
enumerators
}
```

Avoid it: GCC14 documents -Wimplicit-fallthrough=3 in -Wextra and -Wswitch in -Wall. A default label suppresses missing-enumerator warnings from -Wswitch; -Wswitch-enum can warn even with default. Author guidance: use explicit returns/breaks; the GNU fallthrough attribute can mark a deliberate fall-through.

C-PP-07 Signal flag without volatile sig_atomic_t

SR-404 · Ignition-off handler

✕ BUGGY

```
static int stop; // invalid asynchronous flag object
static void on_alarm(int sig) { (void)sig; stop = 1; }
void run_loop(void) {
    signal(SIGALRM, on_alarm);
    alarm(1);
    while (!stop) { }  // invalid async sharing; no
required C outcome
}
```

▸ `hangs: killed by the test watchdog`

What goes wrong: The asynchronous handler's assignment to this plain static int violates C11's signal-handler object restrictions. A timeout is one bounded host observation, not a required optimized outcome or a bug restricted to optimized builds. The POSIX SIGALRM/alarm fixture is not portable ISO C.

✓ FIX

```
static volatile sig_atomic_t stop;
static void on_alarm(int sig) { (void)sig; stop = 1; }
int run_loop(void) { // POSIX, single invocation on this
host
    if (signal(SIGALRM, on_alarm) == SIG_ERR) return -1;
    (void)alarm(1);
    while (!stop) { } // volatile flag; delivery time is
not guaranteed
    return 0;
}
```

Avoid it: For a simple asynchronous flag, C11 permits assigning a volatile sig_atomic_t; other handler actions have separate restrictions. The fixed example checks signal installation and waits for actual delivery, without a one-second scheduling guarantee. Volatile does not provide general thread synchronization or a portable memory-mapped-device protocol.

Sources: C11 draft 7.14; 7.14.1.1p5; 6.7.3; POSIX alarm DESCRIPTION; RETURN VALUE; APPLICATION USAGE; POSIX signal rules 2.4 Signals; POSIX signal DESCRIPTION; RETURN VALUE; APPLICATION USAGE

C-PP-08 Missing #include (implicit declaration)

SR-404 · Dashboard hello.c (Ep 1)

✕ BUGGY

```
int main(void) {
    printf("Hello, World!\n");   // no #include <stdio.h>
    return 0;
}
```

▸ `gcc error: implicit declaration of function 'printf'`

What goes wrong: C11 requires a declared function designator and a diagnostic for a constraint violation; it does not prescribe GCC's warning severity or require translation to stop. GCC14 documents an error by default for implicit function declarations in C99 and later modes, with options that can downgrade it.

✓ FIX

```
#include <stdio.h>
int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Avoid it: Author guidance: include the header declaring each library function, here stdio.h for printf, instead of guessing a prototype. Select the language mode and warning policy explicitly. GCC's -Werror=implicit-function-declaration requests an error for that diagnostic; it is a compiler option, not an ISO C rule.

Sources: C11 draft 5.1.1.3; 6.5.1; 7.21.6.3; GCC14 docs -Wimplicit-function-declaration; GCC14 docs Implicit function declarations

C-PP-09 Missing semicolon

SR-404 · Dashboard (Ep 1)

✕ BUGGY

```
int main(void) {
    printf("Hello, World!\n")    // missing ;
    return 0;  // shown GCC diagnostic is near return
}
```

▸ `gcc error: expected ';' before 'return'`

What goes wrong: A C expression statement ends with a semicolon. Here the missing semicolon makes the source syntactically invalid. Diagnostic wording and location depend on the implementation; a later-line diagnostic can be a clue to an earlier missing token, not a universal parser rule.

Sources: C11 draft 5.1.1.3; 6.8.3; 6.7

✓ FIX

```
int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

Avoid it: Author guidance: start with the first diagnostic and inspect nearby preceding syntax. Struct declarations also need their terminating semicolon. Formatting can help inspection but does not prove syntax or intended behavior. The shown supplied example is checked separately from the game's runtime path.

C-PP-10 #if on an undefined macro

SR-404 · Debug trace build

✕ BUGGY

```
#if KWH_DEBUG_LVL > 1          // undefined here: replaced
with 0
#define TRACE(msg) fputs(msg, stderr)
#else
#define TRACE(msg) ((void)0)
#endif
```

▸ `gcc -Wundef: 'KWH_DEBUG_LVL' is not defined, evaluates to '0'`

What goes wrong: After macro replacement and defined processing in a #if expression, remaining identifiers become preprocessing-number0. With KWH_DEBUG_LVL absent this selects the disabled trace branch. GCC can warn about the undefined identifier; silence is not guaranteed for every build.

Sources: C11 draft 6.10.1p4; GCC14 preprocessor options -Wundef; -dM; -E

✓ FIX

```
#ifndef KWH_DEBUG_LVL
#define KWH_DEBUG_LVL 0         // explicit, documented
default
#endif
#if KWH_DEBUG_LVL > 1           // now a deliberate choice
/* ... TRACE definitions as before ... */
```

Avoid it: GCC14 documents -Wundef and -dM with -E. Author guidance: choose either a numeric-level policy or a defined/absent policy. #ifdef tests existence, so a macro defined as0 still passes it. Defaults can be intentional, but required configuration can instead be rejected with #error.

C-PP-11 Misleading indentation / missing braces

SR-404 · Bus frame handler

✕ BUGGY

```
void on_frame(int crc_ok) {
    if (crc_ok)
        apply_frame();
        log_frame();     // looks guarded, always runs
}
```

▸ `gcc -Wmisleading-indentation: this 'if' clause does not guard...`

What goes wrong: An unbraced if controls its single following statement, regardless of indentation. For crc_ok==0 the supplied bad function still calls log_frame. This is a language-rule demonstration; no historical security incident is required to establish it.

> FIX

```
void on_frame(int crc_ok) {
    if (crc_ok) {
        apply_frame();
        log_frame();
    }
}
```

Avoid it: Author guidance: brace conditional bodies and format consistently. GCC14 documents -Wmisleading-indentation in -Wall for selected misleading layouts, with layout/preprocessor limitations. A clean diagnostic run does not establish that every intended statement is guarded.

Sources: C11 draft 6.8.4.1; GCC14 docs -Wmisleading-indentation

C-PP-12 Inverted or mis-negated condition

SR-404 · Cargo top-box lock (side quest)

✕ BUGGY

```
int topbox_lock_cmd(int key, int unlock_req, int
latch_closed) {
    if (key && unlock_req && latch_closed == 0)  // needs
it open!
        return 1;                                // 1 =
unlock
    return 0;
}
```

▸ `prints: cmd=0`

What goes wrong: For the stated policy, unlocking requires key, request and a closed latch. The bad comparison rejects that all-true input. De Morgan's Boolean identity is !(a&&b) == (!a||!b), but arbitrary C rewrites also need review of side effects and evaluation order. The cargo story is a game model.

> FIX

```
int topbox_lock_cmd(bool key, bool unlock_req, bool
latch_closed) {
    if (key && unlock_req && latch_closed)       // closed
+ request
        return 1;
    return 0;
}
```

Avoid it: Author guidance: use clear predicates and check all eight Boolean combinations against an explicit policy. bool parameters normalize scalar values; they do not validate a real sensor or authorization. Static analysis can detect selected logic issues, but diagnostic scope depends on tool and specification.

Sources: C11 draft 6.3.1.2; 6.5.3.3; 6.5.13; 6.5.14

C-PP-13 Global variable defined in a header

SR-404 · Engine computer modules (Ep 10)

✕ BUGGY

```
/* speed.h */
int speed_kph;  // tentative definition; also a declaration

/* dash.c and cruise.c both #include "speed.h" */
```

▸ `ld: multiple definition of `speed_kph'`

What goes wrong: At file scope int speed_kph is a tentative definition, and also a declaration. In each translation unit without another definition it behaves as a zero-initialized definition. Placing it in both units violates the external-definition requirement. GCC10's porting guide documents the default change to -fno-common; -fcommon is an extension workaround.

> FIX

```
/* speed.h */
extern int speed_kph;          // declaration only

/* speed.c: exactly one definition */
#include "speed.h"
int speed_kph = 0;
```

Avoid it: Author guidance: place an extern declaration in the header and one external definition in a source file. An extern declaration with an initializer would itself be a definition. File-scope static gives internal linkage and separate objects per translation unit when placed in a header; it is not a shared-global repair.

Sources: C11 draft 6.9p5; 6.9.2; 6.2.2; GCC10 porting guide Default to -fno-common

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.